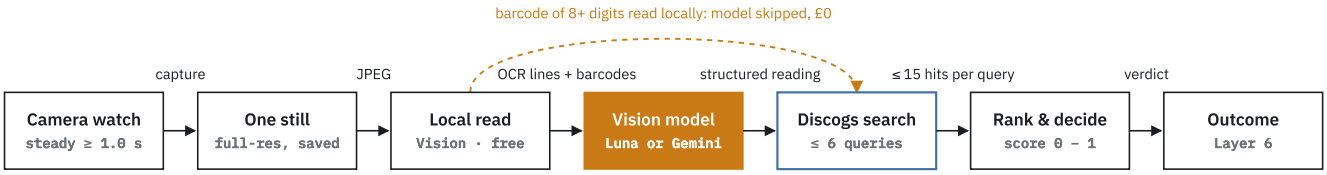
VINYLCAT · SCAN PIPELINE · WORKING TREE AS OF 6 SEP 2026

HOW VINYLCAT FINDS A RECORD

You hold a record up to the camera. About two seconds later the app says the artist, title, year and format out loud, and the record is in the library. This page walks through the six layers that make that happen, what each one costs, and the exact words we send to the vision model.

01 **CAMERA WATCH** Decide when the record is steady and take one still	02 **LOCAL READ** Free, on-device barcode + text recognition	03 **VISION MODEL** Luna or Gemini turns the photo into structured fields
04 **DISCOGS SEARCH** Up to six queries, strongest key first	05 **RANK & DECIDE** Score every hit, then match, ask, or give up	06 **WHAT HAPPENS NEXT** Save, ask you, re-shoot, or park for review

THE WHOLE RUN IN ONE LINE



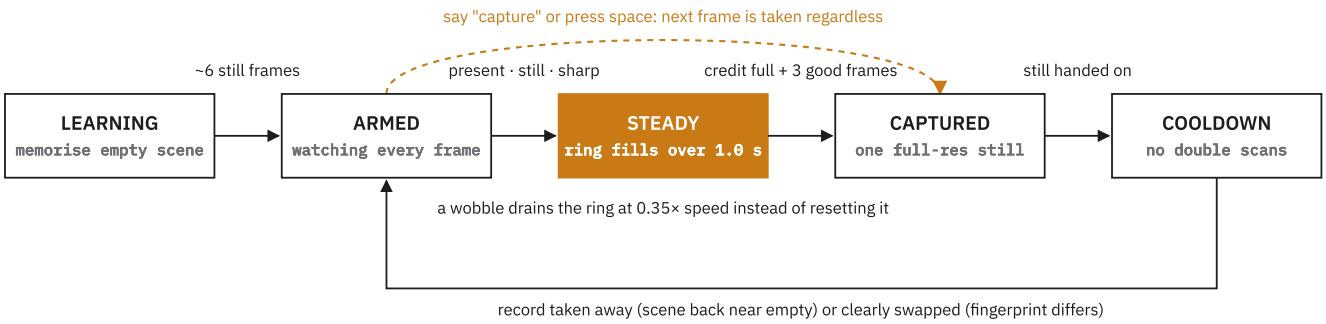
Every scan runs left to right. Only one stage costs money (the vision model) and only one talks to Discogs. A clean barcode lets the run skip the paid stage entirely.

Two ideas underpin the whole design. First, **the catalogue number does most of the work.** A label that says "POSP 113" pins a release far more precisely than the artist and title do, so every layer is tuned to find and preserve that string. Second, **do the free thing first.** Apple's on-device recognition runs before anything is sent to a paid model, and a good barcode means nothing is sent at all.

1 CAMERA WATCH

`Capture/CaptureEngine.swift · Capture/FrameAnalysis.swift`

Nothing is "searched" yet. This layer's only job is to notice that a record is being held still and sharp in front of the camera, and to take exactly one photo of it. It never fires while you are moving the record, and it never fires twice on the same record.



The engine is a small state machine. The only way from ARMED to CAPTURED is a full second of good frames, or you forcing it.

WHAT IT MEASURES

Every camera frame is shrunk to a 96 × 72 greyscale thumbnail of the *central* region only (the alignment guide), so hands at the edges do not count. Four numbers are computed per frame, each smoothed as the median of the last five so a single flicker cannot flip the state:

SIGNAL	HOW	THRESHOLD
Sharpness	Variance of a Laplacian edge filter. Blur flattens edges.	≥ 60
Motion	Mean pixel difference vs the previous frame (0–255).	≤ 6
Presence	Mean difference vs the memorised empty scene.	≥ 14
Fingerprint	64-bit perceptual hash (8 × 8 average hash). "Is this the same item I just shot?"	≤ 10 bits = same

DETAILS THAT MATTER

- **Hold credit, not a timer.** Good frames add time to a 1.0 s bucket; bad frames drain it at about a third of the rate. Once a hold has started, thresholds relax by 50 % so the ring does not strobe.
- **Drift check.** A record sliding past looks "still" frame to frame, so the engine also compares against the frame the hold started from. Three consecutive frames more than 18 apart restart the hold.

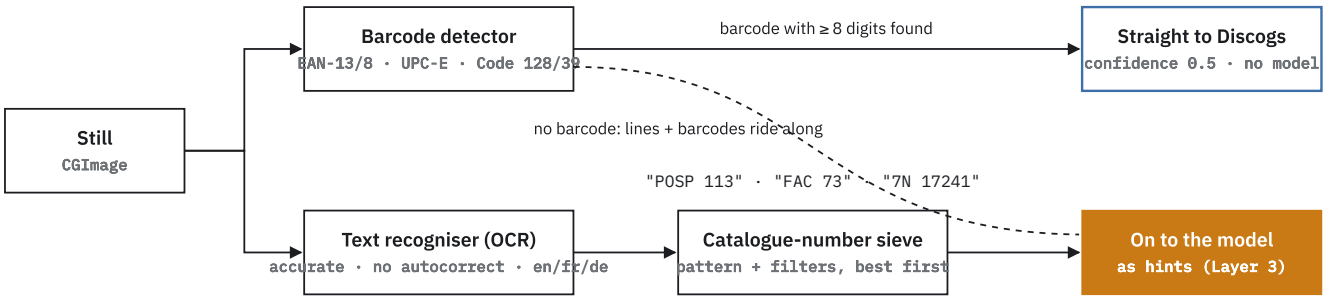
- **Two modes.** `identify` (first shot of a new record) waits for the record to be removed before re-arming. `shot` (extra photos of a matched record) re-arms at once but demands a burst of motion first, so a flip is required before the next capture.
- **Hand-held phone.** The motion limit of 6 and drift limit of 18 were raised specifically so a Continuity-Camera iPhone in the hand can ever fill the ring.
- **Settings sliders** override sharpness, steady seconds and presence thresholds.

---

2 LOCAL READ

Readers/LocalReader.swift · Readers/Identifier.swift

The still goes first to Apple's on-device Vision framework. This costs nothing, works offline, and takes a fraction of a second. It produces three things: any barcodes, every line of printed text it can see, and a shortlist of tokens that look like catalogue numbers.



Barcode and OCR run together on the same still. A confident barcode is the one case where the paid model is never called.

THE CATALOGUE-NUMBER SIEVE

Every OCR line is upper-cased and run through one pattern: up to two short letter/digit groups, then a run of one to seven digits, optionally a second short digit run. That deliberately loose net catches "CHS TT 17", "2059 227" and "SRK 6095", but also years and speeds, so the filters do the real work:

- Drop bare four-digit numbers, "45" and "33" (years and RPM).
- Drop bare numbers shorter than five digits.
- Drop "A1", "B2" style side markers (one digit, fewer than three letters).
- De-duplicate ignoring spaces.
- Sort: anything with letters first, then longer strings first.

The full shortlist, not just the winner, is passed on to the Discogs layer. A label prints matrix and side suffixes too, and the first-ranked token is not always the one Discogs uses.

DETAILS THAT MATTER

- **OCR autocorrect is off.** Apple's language correction would "fix" catalogue numbers into words. We want the raw glyphs.
- **Comparison form.** Whenever two catalogue numbers are compared later, both are reduced to upper-case letters and digits only, so "POSP-113", "POSP 113" and "posp113" all agree.
- **Barcode shortcut** is the setting `skipModelWhenBarcode` (on by default). It needs at least 8 digits, so a stray Code 39 fragment does not trigger it.

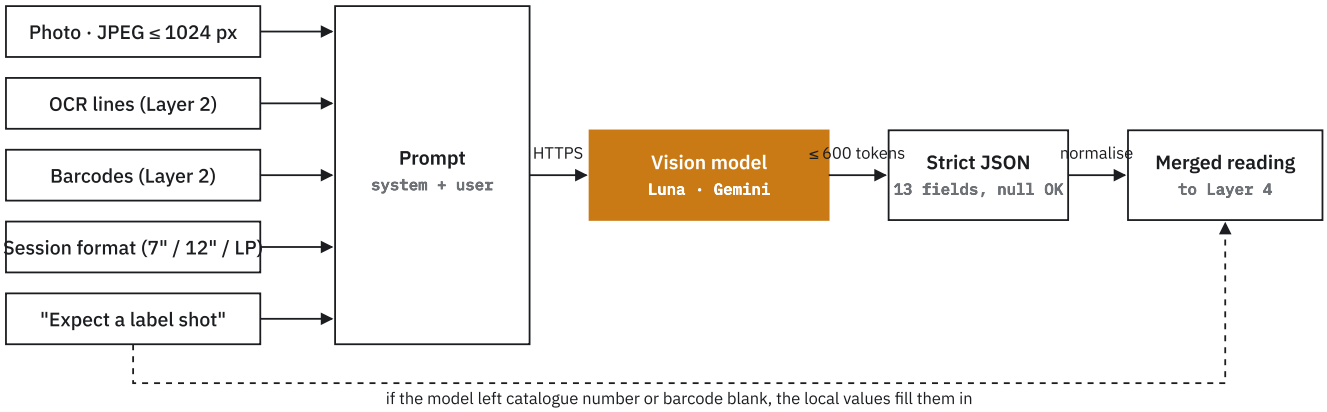
- **No API key at all?** The app still works, badly. It guesses "is this a label?" from words like RPM, SIDE A, MADE IN, ℗, picks the first long non-numeric lines as label / title / artist, sets confidence 0.3, and lets the catalogue number carry the search. The HUD then nudges you to add a key.

---

3 VISION MODEL

Readers/SleeveReader.swift · Readers/LunaReader.swift · Readers/GeminiReader.swift

If there was no usable barcode, the photo goes to a multimodal model with everything the local pass found attached as hints. The model is not asked to identify the release. It is asked to *read* the sleeve or label into thirteen fixed fields, and it is forced to answer in exactly that shape.



The model sees the photo plus the local OCR as a hint, and must answer in a fixed JSON shape. Fields it leaves empty are back-filled from the local pass.

THE EXACT PROMPTS

Both providers get identical wording. The system prompt sets the job and the house rules. The user prompt is assembled per scan from the hints.

SYSTEM PROMPT · SENT VERBATIM, ONE PARAGRAPH

```
You identify vinyl records from a photo of a sleeve (cover) or the paper label in the centre of the disc. Extract only what is visible or can be inferred with high confidence. Catalogue numbers matter most: copy them exactly as printed (e.g. "POSP 113", "FAC 73", "7N 17241"). Do not guess a year unless printed. Set isLabelShot true when the photo shows the round centre label. formatGuess: "7in" for a small single label or 45 RPM marking, "12in" for a 45/33 RPM maxi single, "LP" for an album, "10in" for a ten-inch. confidence is your certainty that artist and title are right (0–1). Return JSON matching the schema; use null for unknown fields.
```

USER PROMPT · ASSEMBLED PER SCAN; LINES 2–4 APPEAR ONLY WHEN THAT HINT EXISTS

```
Read this record photo. The operator is currently scanning seven inch. This should be
the centre label. Barcode detected locally: 5012394112345.
Local OCR lines (may contain errors):
POLYDOR
BLUE MONDAY
POSP 113
45 RPM
SIDE A
```

"The operator is currently scanning seven inch" comes from the format button or the voice commands "sevens", "twelves", "albums". "This should be the centre label" is added after the app has asked you to show the label. The OCR block is a deliberate hint: the model can read the photo itself, but a second opinion on hard-to-read catalogue numbers helps, and the "may contain errors" caveat stops it copying OCR mistakes blindly.

THE ANSWER SHAPE

The model is told it must return every one of these keys (null when unknown). Anything else is rejected by the API before we see it.

FIELD	MEANING
`artist, title`	As printed
`label`	Record company
`catalogNumber`	Exactly as printed. The most important field.
`barcode`	Digits if visible
`year, country`	Only if printed
`formatGuess`	One of 7in · 10in · 12in · LP (app maps to 7", 10", 12", LP)
`isLabelShot`	True for the round centre label
`side`	A / B / 1 / 2 if visible
`visibleTracks`	Track titles it can see
`confidence`	0–1 for artist + title only
`notes`	Free text

PROVIDERS AND COST

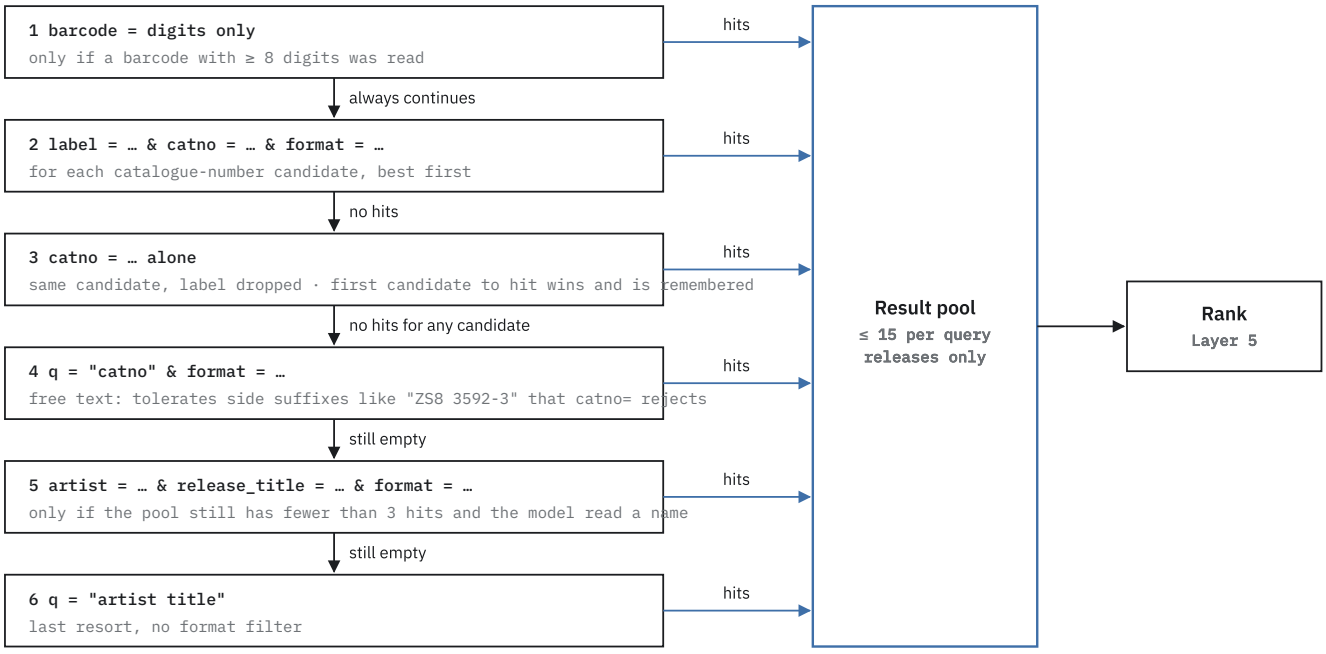
	GPT-5.6 LUNA (DEFAULT)	GEMINI 3.8 FLASH
API	OpenAI Responses	Google generateContent
Output control	JSON schema, strict mode	Response schema, temperature 0.1
Reasoning	effort "low"	none
Image detail	"high"	inline JPEG
Output cap	600 tokens	600 tokens
Price in / out per 1M tokens	$0.20 / $1.20	$0.75 / $3.75
Timeout	45 s	45 s

- **Image size.** The still is downscaled so its longest side is 1024 px and JPEG-compressed at 80 % before upload. That caps the per-image token cost.
- **Cost meter.** Every reply's token counts are multiplied by the price table and added to the session total shown in the HUD.
- **Provider switch** is the Settings choice `readerProvider`. Keys live only in the Keychain.
- **Luna retry.** If OpenAI rejects the "reasoning" or "detail" knobs, the request is sent once more without them.

4 DISCOGS SEARCH

Discogs/Matcher.swift · Discogs/DiscogsClient.swift

Discogs is the only source of truth for what a release actually is. The app never asks Discogs an open question if it can ask a precise one, so it works down a ladder from the strongest key to the weakest, collecting hits into one pool and stopping the moment a precise key lands.



The ladder stops early: a barcode or catalogue-number hit usually ends the run after one or two calls. Free-text and artist/title queries only fire when the precise keys found nothing.

WHAT ACTUALLY GOES OVER THE WIRE

Every search is one HTTPS GET to the same Discogs endpoint. The only things that change between the six rungs are which query parameters are present. Two are always sent: `type=release` and `per_page=15`.

REQUEST HEADERS · EVERY CALL

```
GET https://api.discogs.com/database/search?…
Authorization: Discogs token=<your personal token from Settings>
User-Agent: VinylCat/0.1 +https://github.com/woodall/vinylcat
Accept: application/json
```

PARAMETER	DISCOGS MEANING	WHAT WE PUT IN IT	RUNG
`barcode`	Exact barcode	Digits only, spaces and hyphens stripped: `5012394112345`	1
`catno`	Exact catalogue number as Discogs lists it	The token as read. From the model: as printed on the sleeve (`POSP 113`). From the OCR sieve: upper-cased, spacing as OCR saw it.	2, 3
`label`	Label name	The model's `label` field, untouched: `Polydor`	2
`format`	Format descriptor	Your session choice, or the model's guess: `7"` , `12"` or `LP` (sent URL-encoded as `7%22`)	2, 4, 5
`q`	Free-text search across all fields	Rung 4: the catalogue token alone. Rung 6: `artist` `title` joined with a space.	4, 6
`artist`	Artist name	The model's `artist` field, untouched	5
`release_title`	Release title	The model's `title` field, untouched	5

A worked example. The model reads a Polydor 7" label as artist "New Order", title "Blue Monday", label "Polydor", catalogue number "POSP 113", and OCR also produced "POSP 113" and "45". No barcode. Session format is 7". The calls that go out, in order, until one returns hits:

RUNG 2 · LABEL + CATNO + FORMAT (FIRST HIT ENDS THE LADDER HERE)

```
/database/search?type=release&per_page=15&label=Polydor&catno=POSP%20113&format=7%22
```

RUNG 3 · ONLY IF RUNG 2 WAS EMPTY

```
/database/search?type=release&per_page=15&catno=POSP%20113
```

RUNG 4 · ONLY IF NO CATALOGUE TOKEN HIT AT ALL

```
/database/search?type=release&per_page=15&q=POSP%20113&format=7%22
```

RUNGS 5 AND 6 · ONLY IF THE POOL STILL HAS FEWER THAN 3 HITS

```
/database/search?
type=release&per_page=15&artist=New%20Order&release_title=Blue%20Monday&format=7%22
/database/search?type=release&per_page=15&q=New%20Order%20Blue%20Monday
```

Note what is *not* sent: the photo, the OCR text, your Discogs username, or anything from the model beyond those five fields. Rung 1 would be `barcode=5012394112345` and runs on its own before the catalogue ladder whenever a barcode is available.

After a match, three more calls use the release id from the search hit:

```
FOLLOW-UP CALLS · RELEASE ID 1234567, CURRENCY FROM SETTINGS
GET  /releases/1234567?curr_abbr=GBP                full record: artists, labels,
formats, tracklist, images, genres
GET  /marketplace/stats/1234567?curr_abbr=GBP       lowest price and copies for
sale, in your currency
GET  /oauth/identity                                once, to learn your username
POST /users/<username>/collection/folders/1/releases/1234567    adds it to your
"Uncategorized" folder (if sync is on)
```

Each search reply is read for only these fields per hit: id, title (Discogs formats it "Artist - Title"), year, country, label list, catno, format list, barcode list, thumbnail, cover image and master id. Those are exactly the inputs the ranker in Layer 5 scores against.

DETAILS THAT MATTER

- **Format filter.** Steps 2, 4 and 5 pass the session format (your "sevens / twelves / albums" choice) or, failing that, the model's guess. That is what keeps a 7" single from matching its 12" or album cousin.
- **Which catalogue number won is remembered.** The ranker in the next layer scores against the candidate that actually found the release, not the first thing OCR saw. Otherwise "ZS8 3592-3" read off the label would only ever get an approximate-match score against the "ZS8 3592" Discogs lists.
- **Every query asks for releases only**, never masters, and at most 15 results.
- **Duplicates across queries** are collapsed by Discogs release id before scoring.

RATE LIMIT

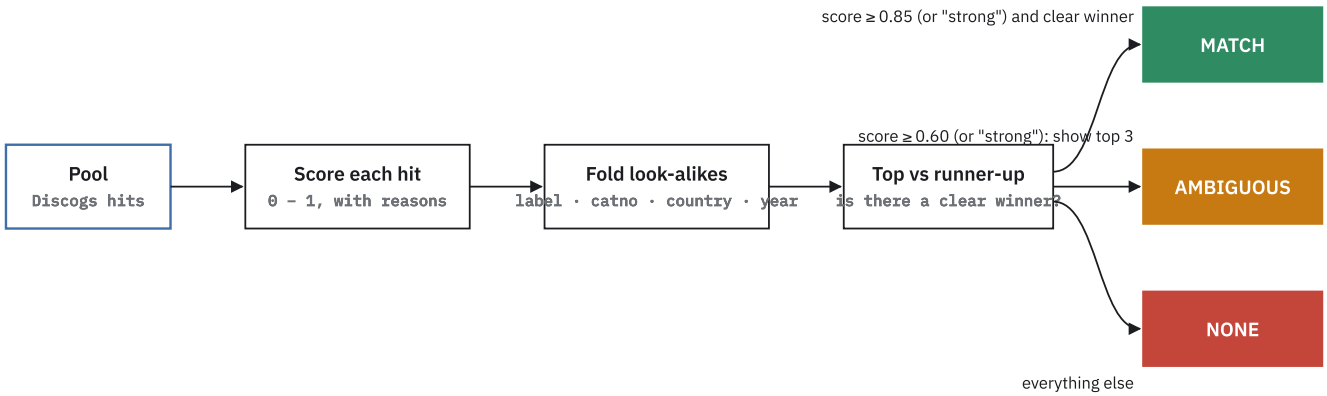
- Discogs allows 60 requests per minute with a token, 25 without. The client reads the limit back from every response header.
- The app keeps five requests of headroom: it never sends more than limit − 5 in any rolling minute, and sleeps for the rest of the minute if it would.
- A 429 (too many requests) backs off for the rest of the window and retries once.
- After a match, two more calls follow in the background: the full release (tracklist, images, genres) and marketplace stats (lowest price, copies for sale, in your currency). A third adds the release to your Discogs collection if sync is on.

---

5 RANK & DECIDE

Discogs/CandidateRanker.swift · Discogs/Matcher.swift

Discogs returns hits, not answers. Each one is scored from 0 to 1 against what was read off the record, look-alike pressings are folded together, and then a simple set of gates decides whether to accept the top hit, ask you to choose, or admit defeat.



Two thresholds, 0.85 to accept and 0.60 to ask, plus a "strong" override for exact catalogue-number or barcode hits whose title also agrees.

SCORING WEIGHTS

Points are added per hit and clamped to 0–1. The reasons are kept and shown in the HUD's meta line (for example `0.92 · r1234567 · catno title 0.96 format`).

EVIDENCE	TEST	POINTS
Catalogue number, exact	Letters + digits identical after normalising	+0.50
Catalogue number, approximate	One contains or ends with the other (4+ chars)	+0.30
Barcode	Digits identical	+0.50
Label	Name similarity above 0.85	+0.15
Artist / title	Similarity above 0.60, scaled	up to +0.25
Format	Matches session or model format	+0.10
Format	Contradicts it	−0.15
Year	Within one year of a printed year	+0.05
Home country	UK if currency is GBP, US for USD, etc.	+0.03

So an exact catalogue number plus a good title plus the right format lands around 0.85 on its own. A barcode plus title does the same. Artist and title alone can never pass 0.85, which is the point: names alone cannot tell pressings apart.

DETAILS THAT MATTER

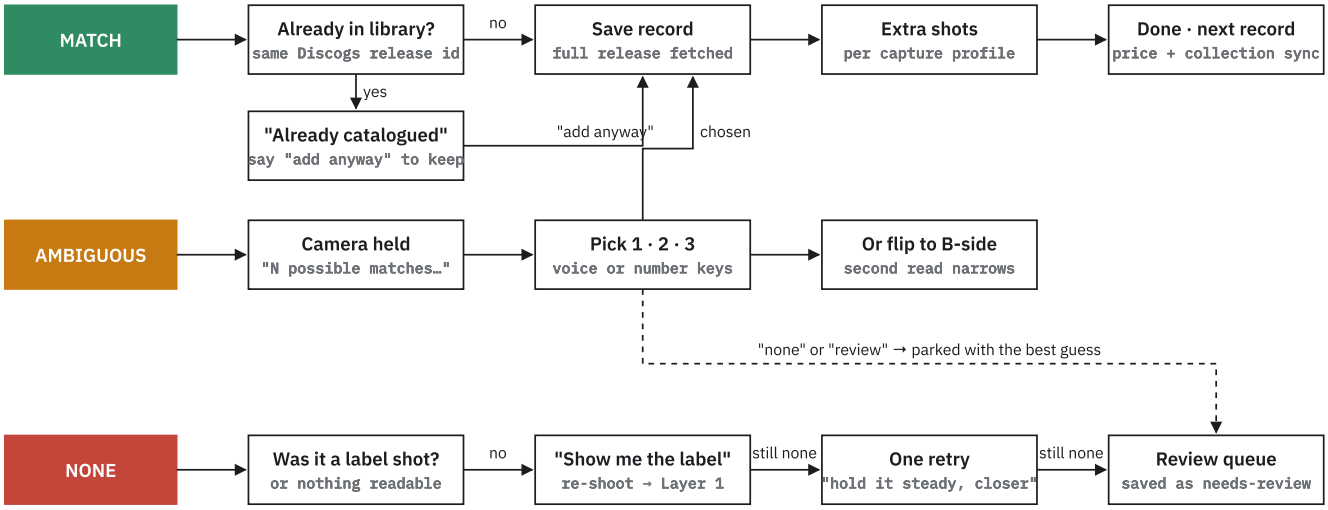
- **Text similarity** is Jaro–Winkler on lower-cased, punctuation-stripped strings with a leading "The" removed. It rewards shared prefixes, which suits titles.
- **Swapped artist and title** are tolerated: the ranker tries the whole "Artist - Title" string and also each part against each part, and takes the best.
- **"Strong"** means an exact catalogue number or barcode *and* a title match. A strong hit is accepted even below 0.85. This is the safety net for scores dented by a wrong format guess.
- **Look-alikes.** Two Discogs entries with the same label, catalogue number, country and year cannot be told apart from a photo, so they are folded into one before deciding. Without this, every reissue would trigger a "which one?" question.
- **Clear winner** means the runner-up is at least 0.10 behind, or both are the same master release and the top is strong.
- **Ambiguous** hands you the top three distinct pressings, each shown as headline plus "label · catno · country · year · format".

---

6 WHAT HAPPENS NEXT

Scan/ScanController.swift · Review/ReviewView.swift

The verdict drives the HUD, the voice, and the camera mode. There are three paths, and two of them loop back to Layer 1 for another photo of the same record before anything is given up on.



Green, amber and red lanes for the three verdicts. Only the red lane ever ends in the review queue, and only after a label shot and one retry.

MATCH LANE

- **Duplicate check** is by Discogs release id. A second copy is allowed, but only if you say "add anyway".
- **What gets saved** comes from the full Discogs release, not the search hit: artist line, title, label, catalogue number, format and disc count, year, country, genres, styles, tracklist, cover image. The photo, the raw OCR text, the barcode and the match score are stored alongside.
- **Extra shots** depend on format. By default singles need none (Discogs already knows the tracklist), albums get front and back cover, gatefolds add a gatefold shot, multi-disc sets add all four labels. Settings can add the B-side for singles and labels for LPs.
- **Spoken confirmation:** "New Order, Blue Monday, 1983, twelve inch." Then the next shot prompt, if any.

AMBIGUOUS AND NONE LANES

- **The camera is frozen** while a question is on screen, so the next steady frame cannot wipe the list. "Capture" and space still work.
- **Second look.** If you flip the record while the list is up, the new read runs the whole pipeline again; if exactly one candidate survives both reads, it is accepted and both photos are kept as sides A

and B.

- **Label first.** When a cover shot fails to match, the app assumes the label will do better, switches the camera to shot mode, and tells the model to expect a label. Only if that also fails does it offer one retry, then park the record.
- **Review tab** keeps everything read so far. You can type extra words; a catalogue number in them is pulled out and searched exactly, the rest goes in as free text, and the same ranker orders the results.

EVERY KNOB IN ONE PLACE

LAYER	SETTING	DEFAULT	WHERE
1	Sharpness minimum	60	Settings slider
1	Steady hold	1.0 s	Settings slider
1	Presence (scene delta) minimum	14	Settings slider
1	Motion maximum / drift maximum	6 / 18	Code only
2	Skip model when barcode ≥ 8 digits	on	Code only
3	Provider	Luna	Settings · `readerProvider`
3	Upload size / output cap	1024 px / 600 tokens	Code only
4	Results per query / Discogs limit headroom	15 / 5	Code only
5	Auto-accept / ask thresholds	0.85 / 0.60	Code only
5	Preferred country	from currency	Settings · `currency`
6	Extra shots (B-side on singles, labels on LPs)	off / off	Settings · `askForBSide`, `askForLabelsOnLP`
6	Add matches to Discogs collection	on	Settings · `discogsSync`

Sources (working tree, including uncommitted edits): `Sources/VinylCat/Capture/CaptureEngine.swift`, `Capture/FrameAnalysis.swift`, `Readers/LocalReader.swift`, `Readers/Identifier.swift`, `Readers/SleeveReader.swift`, `Readers/LunaReader.swift`, `Readers/GeminiReader.swift`, `Discogs/Matcher.swift`, `Discogs/CandidateRanker.swift`, `Discogs/DiscogsClient.swift`, `Scan/ScanController.swift`, `Review/ReviewView.swift`, `Persistence/Models.swift` (CaptureProfile).